paxmeet_icons

Custom icon-font package — Documentation

Install & usage · Adding / renaming / removing icons

Using paxmeet_icons (Flutter app + Web)

One icon set, defined once, used the same way everywhere. The names are **identical** across platforms, so designers and developers refer to the same list (open `index.html` to browse & search every icon).

Platform	How you use an icon
Flutter	`Icon(PaxmeetIcons.home)`
Web (React/Next.js)	`<PaxmeetIcon name="home" />` or `<i className="pmi pmi-home" />`

Both render from the **same font file** (`fonts/PaxmeetIcons.ttf`), so an icon can never look different between the app and the website.

A) Flutter app

1. Add the dependency

In your app's `pubspec.yaml` under `dependencies:` — pick one:

```
# Git (recommended — works in any project)
paxmeet_icons:
  git:
    url: https://github.com/letssuhail/paxmeet_icons.git
    ref: main          # or a tag like v0.0.1 to pin a version

# …or local path (same repo / monorepo)
# paxmeet_icons:
#   path: ../paxmeet_icons
```

2. Install

```
flutter pub get        # this repo uses fvm:  fvm flutter pub get
```

3. Use

```
import 'package:paxmeet_icons/paxmeet_icons.dart';

Icon(PaxmeetIcons.home);
```

```
Icon(PaxmeetIcons.heart, size: 28, color: Theme.of(context).primaryColor);

IconButton(
  icon: const Icon(PaxmeetIcons.search),
  onPressed: () {},
);
```

That's it — no font setup in your app's `pubspec.yaml`; the font ships inside the package. The color comes from Flutter (`color:`), the font is monochrome.

> **App size:** all icons are ~12 KB and Flutter **tree-shakes** the ones you don't use out of release builds automatically. Just always use the constants directly (`PaxmeetIcons.home`) — never build `IconData` from a variable codepoint, or tree-shaking turns off.

B) Web (Next.js / React)

Install straight from GitHub — just like the Flutter dependency, no manual file copying. The consumer needs **read-access to the (private) repo** (their git/npm must be authenticated to GitHub).

1. Install from GitHub

```
npm install github:letssuhail/paxmeet_icons
```

2. Let Next.js transpile the package — in `next.config.mjs`

```
const nextConfig = {
  transpilePackages: ['paxmeet_icons'],
};
```

3. Load the CSS once (App Router → `src/app/layout.js`)

```
import "./globals.css";
import "paxmeet_icons/css";
```

4. Use it

```
import { PaxmeetIcon } from "paxmeet_icons";

<PaxmeetIcon name="search" size={20} color="#7332D6" />
```

Or with plain CSS classes (no component):

```
<i className="pmi pmi-search" style={{ fontSize: 20, color: "#7332D6" }} />
```

`paxmeet_icons/names` exports `paxmeetIconNames` — the full list, handy for dropdowns.

Updating later: `npm update paxmeet_icons` (or reinstall) pulls the latest from `main`.

> **TypeScript site?** Props are `{ name: string; size?: number; color?: string; className?: string }`.
>
> **No GitHub access / prefer not to install?** You can instead copy the package's `web/` folder into your project (`cp -r paxmeet_icons/web/* src/components/paxmeet-icons/`) and import from that local path. The git-install above is the cleaner option.

Finding icon names

- Open **`index.html`** (repo root) in a browser — search any icon, click to copy its name / Flutter code. Host it on GitHub Pages to share a public reference.
- Names are camelCase and the same on both platforms: `addCircle`, `wallet`, `volume`, `profileCircle`, `chevronLeft`, …

See **ADDING_ICONS.md** to add or change icons.

Adding / updating / removing icons (updates app + web together)

You edit **one** folder of SVGs and run **one** command. That regenerates the font, the Flutter class, **and** the web assets — so the app and website always get the same icons. The single source of truth is `iconsvgs/` — a flat folder where **the filename is the icon name**.

```
iconsvgs/home.svg        ┐                    ┌─▶ lib/paxmeet_icons.dart   (Flutter: PaxmeetIcons.h
iconsvgs/heart.svg       ├─ tool/build.sh ─▶  ├─▶ web/paxmeet-icons.css     (Web: <PaxmeetIcon name=
iconsvgs/search.svg      ┘   (one font)       └─▶ index.html, preview.png  (reference)
```

One-time setup (first time on a machine)

```
cd paxmeet_icons

# Python tools: picosvg (stroke→fill normalization) + markdown (for the PDF)
python3 -m venv tool/.venv
tool/.venv/bin/pip install picosvg markdown

# Dart tool: the font generator
dart pub global activate icon_font_generator
```

Add a new icon

1. **Prepare the SVG**
2. **Single color** (monochrome) — the color is applied later (`Icon(color:)` in Flutter, `color` in CSS). Multi-color/brand icons can't be a font glyph.
3. Fill-based **or** stroke-based both work — strokes are auto-converted to filled outlines by picosvg.
4. Recommended: `24×24` viewBox, no embedded raster images.
5. **Name it well** and drop it into `iconsvgs/`
6. Use **snake_case**; it becomes a **camelCase** name on both platforms:
   - `event_ticket.svg` → Flutter `PaxmeetIcons.eventTicket` / Web `name="eventTicket"`
7. Name by **what the icon depicts** or its UI meaning — `wallet` not `empty_wallet`, `message` not `message_text`, `microphone` not `microphone_2`.
8. **Build everything (one command)** `bash bash tool/build.sh` This updates the font, Flutter class, web CSS/component, and the reference page.
9. **Commit & push** so apps and sites can pull it `bash git add -A && git commit -m "icons: add eventTicket" && git push`
10. **Pick it up in each project**
11. **Flutter app:** `flutter pub get` (or `fvm flutter pub get`) + full restart.
12. **Website:** re-copy `web/*` into the site (`cp paxmeet_icons/web/* <site>/src/components/paxmeet-icons/`).

Rename an icon

1. Rename the file in `iconsvgs/` (e.g. `empty_wallet.svg` → `wallet.svg`).
2. `bash tool/build.sh` (the build clears stale names automatically).
3. Update usages: Flutter `PaxmeetIcons.emptyWallet` → `PaxmeetIcons.wallet`; Web `name="emptyWallet"` → `name="wallet"`.
4. Commit, push, update each project.

Remove an icon

1. Delete the file from `iconsvgs/`.
2. `bash tool/build.sh`, commit, push.

What each file/script is

File / folder	Role
`iconsvgs/`	**Source of truth** — flat, clean-named SVGs you edit.
`tool/build.sh`	**The one command** — runs the whole pipeline below.
`tool/import.sh`	Normalizes SVGs with picosvg → `tool/svgs/` (clears old files).
`tool/generate.sh`	`icon_font_generator` : `tool/svgs/` → `.ttf` + `lib/paxmeet_icons.dart`.
`tool/build_web_assets.py`	Generates `web/` CSS + React component from the font.
`tool/build_web.py`	Generates the `index.html` reference page.
`tool/preview.py`	Generates `preview.png`.
`fonts/PaxmeetIcons.ttf`	The generated font — used by **both** app and web (committed).
`lib/paxmeet_icons.dart`, `web/*`	Generated outputs (committed).

Gotchas

- **Empty / wrong glyph** → the SVG was multi-color or had an embedded image. Flatten it to a single-color vector and rebuild.
- `import.sh` **prints "PICOSVG FAILED"** for a file → picosvg can't resolve it (multi-color/raster). Simplify and re-export.
- Always commit the regenerated `fonts/`, `lib/paxmeet_icons.dart`, and `web/` together with the SVG change, so app and web stay in sync.
- After updating, **restart** the Flutter app fully (not just hot reload) and re-copy `web/*` into the website.